Pongpipat Kookitwattana 6738160021

Exploratory Data Analysis

```
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              27106 non-null  int64
 1   name                            26557 non-null  object
 2   host_id                         27106 non-null  int64
 3   host_name                       26552 non-null  object
 4   neighbourhood_group             26566 non-null  object
 5   neighbourhood                   26571 non-null  object
 6   latitude                        27106 non-null  float64
 7   longitude                       27106 non-null  float64
 8   room_type                       26570 non-null  object
 9   price                           26595 non-null  float64
 10  minimum_nights                  26586 non-null  float64
 11  number_of_reviews               26583 non-null  float64
 12  last_review                     21111 non-null  object
 13  reviews_per_month               21340 non-null  float64
 14  calculated_host_listings_count  26586 non-null  float64
 15  availability_365                26583 non-null  float64
dtypes: float64(8), int64(2), object(6)
```

We can see that there are a lot of missing information. Because there should be 27106 rows with information.

```
Listings with price <= 0: 547
Listings with minimum_nights < 0: 542
Listings with number_of_reviews < 0: 545
Listings with reviews_per_month < 0: 539
Listings with calculated_host_listings_count < 0: 542
Listings with availability_365 < 0: 541
```

	id	host_id	latitude	longitude	price	minimum_nights	number_of_reviews	reviews_per_month	calculated_host_l
count	27106.000000	2.710600e+04	27106.000000	27106.000000	26595.000000	26586.000000	26583.000000	21340.000000	
mean	13692.979156	6.817564e+07	40.728885	-73.952079	164.604200	7.079687	21.255378	1.379605	
std	7905.881696	7.911806e+07	0.054639	0.046297	6772.488296	614.394492	1369.984414	58.940016	
min	1.000000	2.438000e+03	40.506410	-74.242850	-44039.906822	-3995.969493	-8836.739029	-341.215943	
25%	6851.250000	7.958923e+06	40.690022	-73.983117	66.000000	1.000000	1.000000	0.180000	
50%	13688.500000	3.104429e+07	40.723105	-73.955720	107.000000	3.000000	5.000000	0.730000	
75%	20541.750000	1.074344e+08	40.763098	-73.935830	180.000000	5.000000	25.000000	2.170000	
max	27379.000000	2.743213e+08	40.913060	-73.717950	44388.408761	4008.347956	8875.024307	343.979641	

There are a lot of negative numbers on the information which we know it is not possible such as price , minimum nights. Also the maximum avaibility should not exceed 365 but here it shows that maximum or “availibity” column is 26518 nights.

```
neighbourhood
Williamsburg          7.6625
Bedford-Stuyvesant    7.3379
Harlem                5.1317
Bushwick              4.9251
Upper West Side       3.9585
                       ...
_EXOTIC_37            0.0037
_EXOTIC_5             0.0037
_EXOTIC_0             0.0037
_EXOTIC_43            0.0037
_EXOTIC_12            0.0037
Name: proportion, Length: 284, dtype: float64
```

```
neighbourhood_group
Manhattan      43.3151
Brooklyn       40.1756
Queens         11.3665
Bronx           2.1730
NaN             1.9922
                ...
_EXOTIC_16      0.0037
_EXOTIC_33      0.0037
_EXOTIC_26      0.0037
_EXOTIC_61      0.0037
_EXOTIC_1       0.0037
Name: proportion, Length: 73, dtype: float64
```

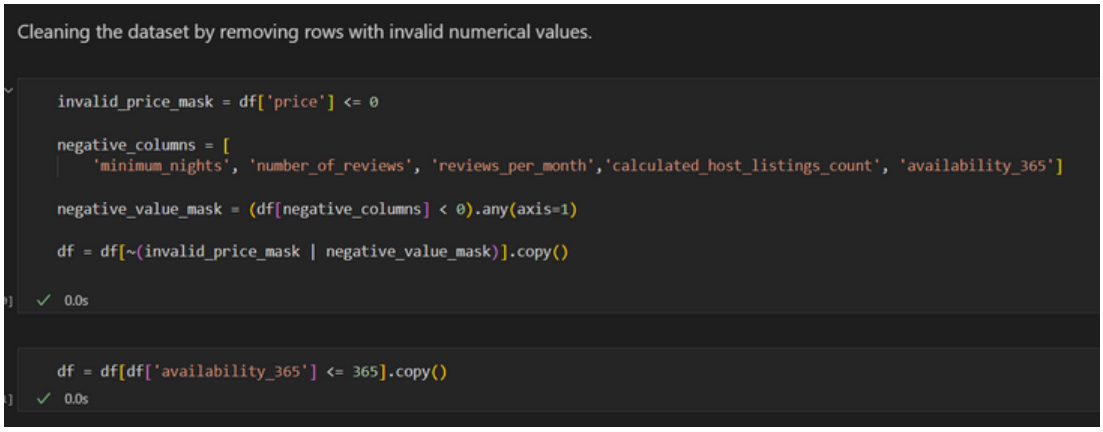
```
room_type
Entire home/apt    50.9186
Private room       44.5621
Shared room         2.2910
NaN                 1.9774
_EXOTIC_17          0.0037
                    ...
_EXOTIC_5           0.0037
_EXOTIC_32          0.0037
_EXOTIC_33          0.0037
_EXOTIC_45          0.0037
_EXOTIC_16          0.0037
```

After checking for negative value of numerical columns, now we should check for non relate value of categorial column. Using value counts we see that “EXOTIC” are not information we need and it only show up a few times on our data.

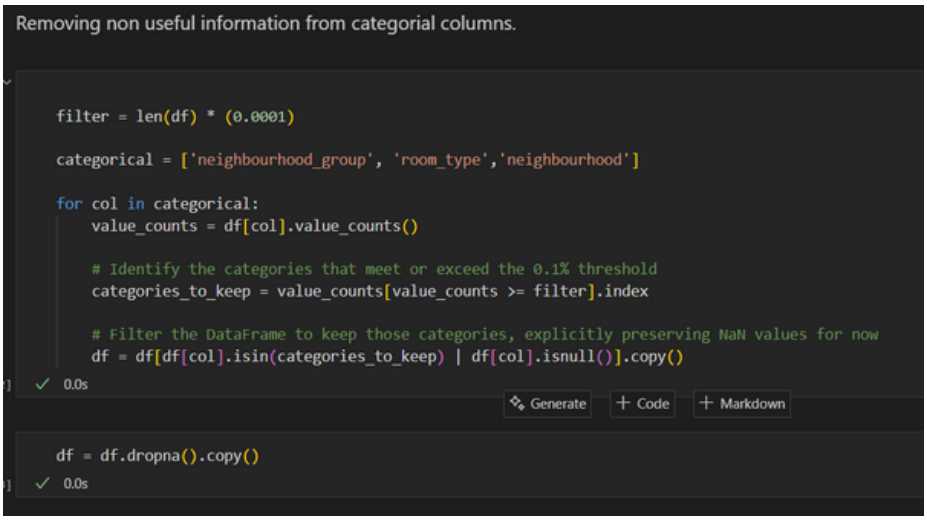
	price
count	26595.000000
mean	164.604200
std	6772.488296
min	-44039.906822
25%	66.000000
50%	107.000000
75%	180.000000
max	44388.408761

For many numerical columns , mean and max is very far off , this mean there are outliers and we should remove them using IQR before use the data for training.

Data Preprocessing



Cleaning negative value and invalid value such as negative price or avaibility more than 365 days.



Removing non useful information from categorial columns
I use filter as 0.0001*len(df) because its mean that if any unique value appear less than 0.01% of the data then we will remove it.



Dropping non useful columns. Because I dont think these columns can be use to train model well.

Removing Outliers using IQR.

markdo

```
numerical_cols_iqr = ['price', 'minimum_nights', 'number_of_reviews', 'reviews_per_month','calculated_host_listings_count', 'availability_3

outlier_mask = pd.Series(True, index=df.index)

for col in numerical_cols_iqr:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    col_mask = (df[col] >= lower_bound) & (df[col] <= upper_bound)
    outlier_mask = outlier_mask & col_mask

df_no_outliers = df[outlier_mask].copy()
```

✓ 0.0s Pyth

Removing Outliers using IQR method.

Train Test Split

Generate + Code + Markdown

```
X = df_no_outliers.drop('price', axis=1)
y = df_no_outliers['price']
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.3,random_state=67381600)
```

[166] ✓ 0.0s

Generate + Code + Markdown

Scaling and Encoding using Scikit Learn Pipilines

```
numerical_features = X_train.select_dtypes(include=np.number).columns.tolist()
categorical_features = X_train.select_dtypes(include='object').columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
        ('num', StandardScaler(), numerical_features)
    ],
    remainder='passthrough'
)

X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

[167] ✓ 0.0s

Train Test Split and Scaling and Encoding using Scikit Learn

Model Building

```
rfr_model = RandomForestRegressor(n_estimators = 200,  max_depth = 13, min_samples_split = 25,random_state = 67381600, n_jobs=-1)
rfr_model.fit(X_train, y_train)

y_pred_rfr = rfr_model.predict(X_test)

rfr_mse = mean_squared_error(y_test, y_pred_rfr)
rfr_r2 = r2_score(y_test, y_pred_rfr)
```

Fit and predict Random Forest Regressor Model

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error, r2_score, make_scorer

r2_scorer = make_scorer(r2_score)

param_grid = {
    'alpha': np.logspace(-4, 0, 20)
}

lasso = Lasso(random_state=67381600, max_iter=1000)

grid_search = GridSearchCV(
    estimator=lasso,
    param_grid=param_grid,
    scoring=r2_scorer,
    cv=5,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
best_lasso_model = grid_search.best_estimator_
best_params = grid_search.best_params
```

Fit and predict Lasso Model

Data Visualisation

Random Forest Features Importance

Feature	Importance
cat__room_type_Entire home/apt	0.522903
num__longitude	0.159442
num__latitude	0.109713
num__availability_365	0.066140
num__reviews_per_month	0.035479
num__number_of_reviews	0.027206
num__minimum_nights	0.016167
cat__neighbourhood_Williamsburg	0.008703
num__calculated_host_listings_count	0.005818
cat__neighbourhood_group_Manhattan	0.004693

Lasso Model Coefficient

Feature	Coefficient
cat__room_type_Entire home/apt	75.163780
cat__neighbourhood_group_Staten Island	-47.451177
cat__neighbourhood_Springfield Gardens	45.198645
cat__neighbourhood_Inwood	-44.839246
cat__neighbourhood_Brooklyn Heights	40.614940
cat__neighbourhood_Washington Heights	-40.228194
cat__neighbourhood_group_Manhattan	37.985932
cat__neighbourhood_South Slope	32.853559
cat__neighbourhood_Downtown Brooklyn	29.614811
cat__neighbourhood_Tribeca	28.835874

Evaluation

```
--- Lasso Model Evaluation ---
Mean Squared Error (MSE): 2261.61
R-squared (R2): 0.4914
```

```
--- Random Forest Model Evaluation ---
Mean Squared Error (MSE): 2210.20
R-squared (R2): 0.5030
```

Random Forest Model has less MSE and high R2 score compare to Lasso Model.

Random Forest Model is Better, even though I use GridSearch CV for Lasso but did not use for Random Forest.

Personally , I think it is because Lasso is a linear model but I dont think the model for renting house in Newyork is linear , because the data set given is not really cover all property of a house such as how many beds are there or how many bathroom or size of the house which indicate price of the house.